

Relații între modele — exerciții

one-to-one · one-to-many · many-to-many · compoziție vs agregare (Java OOP)

MyCodeSchool

Mini-teorie (2 minute)

O **relație (asociere)** = un obiect ține o referință către alt obiect. Patru forme:

Tip	Exemplu	Cum arată în cod
one-to-one	Persoană <-> Cont	fiecare ține un singur câmp către celălalt
one-to-many	Autor -> Cărți	„unul” ține <code>List<Celălalt></code> ;
many-to-one	Carte -> Autor	„mulții” țin un câmp înapoi e același 1-many privit dinspre partea „mulților”
many-to-many	Studenti <-> Cursuri	ambele țin <code>List<></code> către celălalt

Regula de aur (o folosești la fiecare exercițiu): când relația e în **ambele sensuri**, orice metodă care o creează trebuie să seteze **ambele capete** — altfel ai o carte care „crede” că are alt autor decât autorul care o listează. E sursa #1 de bug-uri la relații.

A doua regulă (din review): un getter de listă returnează o **copie**, nu lista internă.

Nivelul 0 — warm-up: one-to-one

Modele: Persoana, Buletin

Fiecare persoană are exact un buletin; fiecare buletin aparține unei persoane.

Cerință: Persoana ține un câmp Buletin; Buletin ține serie, numar și o referință Persoana. Metoda `atribuie(Buletin b)` pe Persoana leagă ambele capete.

Definition of Done:

- ☐ `Persoana.buletin` și `Buletin.persoana` sunt private
- ☐ `atribuie(Buletin b)` setează și `this.buletin = b`, și `b.setPersoana(this)`
- ☐ `descriere()` pe persoană afișează seria buletinului (traversezi relația)
- ☐ dacă persoana avea deja alt buletin, cel vechi rămâne fără persoană (rupi vechea legătură)

Hint: „setează ambele capete” apare deja aici, la cel mai simplu caz — memorează tiparul, se repetă peste tot.

Nivelul 1 — one-to-many, un singur sens

Modele: Autor, Carte

Cerință: Autor are `List<Carte>`; Carte are doar titlu și `anAparitie` (NU știe încă de autor).

Metode: `adaugaCarte(Carte c)`, `numarCarti()`, `getCarti()`.

Definition of Done:

- ☐ `carti` e `private List<Carte>`, inițializat la declarare
- ☐ `adaugaCarte` refuză `null` și duplicatele
- ☐ `getCarti()` returnează o **copie**
- ☐ `main`: un autor cu 3 cărți, afișezi câte are

Hint: e chiar structura `Comanda` -> `listaLinii` din proiectul tău. Relația merge deocamdată doar dinspre autor spre cărți.

Nivelul 2 — one-to-many bidirecțional (aici e miezul)

Modele: Echipa, Jucator

Acum relația merge în **ambele** sensuri: echipa își știe jucătorii, jucătorul își știe echipa.

Cerință: Echipa are `List<Jucator>`; Jucator are un câmp `Echipa` `echipa`. `transfera(Jucator j)` pe echipă îl adaugă în listă **și** setează `j.echipa = this`.

Definition of Done:

- ☐ `transfera` setează ambele capete (regula de aur)
- ☐ `jucator.getEchipa().getJucatori()` îl conține pe `j` (test de consistență)
- ☐ transferul la altă echipă îl **scoate** din lista echipei vechi înainte de a-l adăuga la cea nouă
- ☐ niciun jucător nu apare în două echipe simultan

Hint: scrie întâi metoda naiv (doar adaugă în listă), rulează testul de consistență și vezi cum pică — apoi adaugă și al doilea capăt. Vrei să *simți* de ce e nevoie de ambele.

Nivelul 3 — many-to-many

Modele: Student, Curs

Un student urmează mai multe cursuri; un curs are mai mulți studenți. **Ambele** țin liste.

Cerință: Student are `List<Curs>`, Curs are `List<Student>`. `inscrie(Student s)` pe curs adaugă în ambele liste.

Definition of Done:

- ☐ `inscrie` adaugă studentul în `curs.studenti` **și** cursul în `student.cursuri`
- ☐ nu poți înscrie același student de două ori la același curs
- ☐ `getStudenti()` / `getCursuri()` returnează **copii**
- ☐ main: 2 studenți, 3 cursuri, câteva înscrieri; afișezi cursurile lui s1 și studenții lui c2

Hint: o singură metodă publică `inscrie` pe **una** dintre clase (ex. `Curs.inscrie(Student)`) care leagă ambele capete; nu duplica logica în ambele clase, altfel te încurci care pe cine actualizează.

Nivelul 4 — compoziție vs agregare (lifecycle)

Modele: Comanda, LinieComanda, Produs

Cerință: Comanda **deține** liniile (compoziție: fără comandă, linia n-are sens -> o creezi în interiorul comenzii). LinieComanda doar **referă** Produs (agregare: produsul există independent, e partajat de mai multe linii).

Definition of Done:

- ☐ Comanda.adaugaLinie(Produs p, int cant) construiește LinieComanda **în interior** (nu primești linia din afară)
- ☐ același Produs poate apărea în linii din comenzi diferite (e partajat, nu copiat)
- ☐ total() traversează liniile și adună subtotalele
- ☐ ștergerea unei comenzi nu „distruge” produsele (sunt independente)

Hint: compoziția = „parte-din, moare cu întregul”; agregarea = „folosește, dar trăiește separat”. Comanda ta din proiect e deja aproape — diferența e conceptuală, formuleaz-o în cuvintele tale.

Provocări (fără hint)

1. **Bibliotecă** — Autor, Carte, Editura. O carte are **un** autor (many-to-one) și **o** editură (many-to-one), iar autorul își știe cărțile (bidirecțional), editura nu. Metodă: `cartileAutoruluiLaEditura(Autor a, Editura e)`.
2. **Rețea socială** — Utilizator cu `List<Utilizator>` prieteni. Prietenia e **simetrică** (many-to-many pe aceeași clasă!): dacă A e prieten cu B, atunci B e prieten cu A. Metode: `imprietenire(Utilizator)` și `prieteniComuni(Utilizator altul)`.
3. **Playlist** — Melodie $\leftrightarrow$ Playlist (many-to-many), plus o metodă care găsește melodiile aflate în **cel puțin 2** playlist-uri.

Capstone — „Școala”

Modele (3-4): Profesor, Curs, Student (+ opțional Nota)

- Profesor 1->many Curs (un profesor predă mai multe cursuri; cursul are un profesor)
- Student many<->many Curs
- (bonus) Nota leagă un Student și un Curs cu o valoare — modelezi relația many-to-many **cu atribut** printr-o clasă intermediară

Definition of Done (final):

- ☐ toate câmpurile private, toate listele returnate ca **copii**
- ☐ fiecare relație bidirecțională e consistentă în ambele capete
- ☐ `mediaStudentului(Student)`, `studentiiCursului(Curs)`, `cursurileProfesorului(Profesor)`
- ☐ un main care populează 2 profesori, 3 cursuri, 4 studenți, câteva note și afișează un raport